

Rank-Two Terminal Contractions and Singularity-Safe Modular Observers

An exact boundary-query improvement for the
ProveIt unknot-recognition program

Research contribution prepared for Vladimir Reshetnikov
ProveIt repository; proofs, reference implementation, and reproducible audits

8 October 2026

Abstract

The maintained ProveIt recognizer has exact boundary-completion observers, but its research determinant kernel evaluates distinct terminal identifications by fresh small-border elimination. We develop an edit-sensitive alternative. A terminal partition is represented by a fixed-size, generally nonsymmetric matrix: block anchors carry sums of original kernel rows, while the other rows impose equality constraints. An anchored block merge changes this matrix by a sum of two explicitly identified rank-one matrices. Its determinant is exactly the quotient cofactor, with no partition-dependent sign.

Over each prime field we first eliminate the graph interior by left–right rank normalization. Singular interiors are retained as null directions rather than excluded as “bad primes.” A deterministic rank-normal-form update handles all rank changes in quadratic field work. Consequently a supplied merge tree with J edges is evaluated in $O(v^3 + Jb^2 + Q)$ field operations, where v is graph size, b is terminal count, and Q is the number of observations. We provide signed Chinese-remainder reconstruction, independently checkable algebraic certificates, transaction-safe persistent cursors, and an elementary polynomial prime-budget bound.

The package passes 39 unit tests, 19,200 matrix updates, 1,772 graph-query comparisons, and all 877 partitions of seven terminals. Seven-round paired finite-field observer benchmarks show up to 3.34-fold improvement against fresh elimination of the smaller singular border; small exact-integer workloads favor direct Bareiss elimination by about an order of magnitude. These are observer measurements, not whole-recognizer speedups. The result improves the cost of related observations but does not bound their number, make the determinant an unknot detector, or establish a general quasi-polynomial recognition algorithm.

Status. Research draft with complete elementary proofs of the stated algebraic results and an executable Python reference package. No formal proof-assistant verification or full upstream test-suite run is claimed. The integration adapter is additive and is not enabled in the maintained recognizer.

Contents

1	Problem, contribution, and research position	4
1.1	The bottleneck selected from the repository	4
1.2	What is proved and what is implemented	4
1.3	Relation to existing mathematics	5
1.4	Notation and complexity model	5
2	Signed graph quotients and a uniform reconstruction bound	5
2.1	Cofactor convention	5
3	A prime-specific singular interior kernel	6
4	Fixed-dimension matrices for all terminal partitions	8
5	An anchored merge is a rank-two update	9
6	Quadratic updates through every singular case	10
6.1	Invariant and determinant bookkeeping	10
6.2	Persistent branching and failure safety	11
7	Edit-sensitive batch bounds and the partition tree	12
7.1	An exact tree on all set partitions	13
7.2	A possible refinement: zero descendant cones	13
8	Signed reconstruction and a deterministic bit budget	14
8.1	Exact reconstruction	14
8.2	Elementary control of the prime supply	14
8.3	Complete bit accounting	15
9	Certificates and the boundary-geometry interface	16
9.1	An independently checkable algebraic certificate	16
9.2	A fully explicit certificate example	16
9.3	What the maintained geometry must still certify	17
9.4	Adapter status and deliberate limitations	17
10	Validation and performance measurements	17
10.1	Correctness checks that were actually run	17
10.2	Benchmark design	18
10.3	Exact arithmetic gives an important negative result	19
10.4	Interpreting performance without overfitting	20
11	Consequences for the quasi-polynomial goal	20
12	Further research questions	21
12.1	Actual-query merge complexity	21
12.2	Adaptive static, integer, and dynamic dispatch	21
12.3	Zero-cone pruning across primes	21
12.4	Reuse between successive suffix stages	22
12.5	Cofacial partition structure and grove identities	22

12.6 Cheaper deterministic evidence	22
12.7 Rank-sensitive and sparse update engines	22
12.8 Sharper reconstruction and early obstruction	22
12.9 From scalar observations to homological compression	22
12.10A global obstruction family or a global structural theorem	23
13 Integration and reproducibility	23
14 Conclusion	23
A Algorithmic contracts in compact form	24
A.1 Prime kernel	24
A.2 Persistent merge	24
A.3 Rank-one branch conditions	24
B Proof and implementation obligation ledger	25

1 Problem, contribution, and research position

1.1 The bottleneck selected from the repository

The repository head recorded during source review was

cbc627dbbba08e3125166447ab3aa1157f9dc42e.

The relevant maintained sources are `synthesis/boundary_reuse.tex` and the boundary observer `boundary_tait.py` under `fast/determinant_research/`. The latter delegates rational linear algebra to the earlier report-26 terminal kernel. The review also examined the main README and the corridor-transfer analysis. Exact paths and reviewed Git blob identifiers are recorded in the accompanying provenance file [1, 2, 3].

The boundary observer isolates a fixed suffix of a diagram. A completed boundary matching can identify several of its cut-face fragments. A checkerboard shading turns the black fragments into a common signed Tait graph, and only designated terminal vertices are identified. Thus many topologically distinct completion queries ask for cofactors of quotients of *one* signed graph. The current research design already saves geometric work and caches identical terminal partitions. The remaining distinct partitions are evaluated by determinant calculations on a small singular-safe border [2].

This report targets that last, algebraic operation. Related partitions should share their elimination work, not merely their graph construction. However, ordinary inverse updates are not sufficient: the interior can be singular, the quotient determinant can vanish, and even an update between two invertible endpoint matrices can pass through a singular intermediate matrix.

1.2 What is proved and what is implemented

There are three substantive interfaces.

Graph to modular border. An integer signed graph and a list of terminals produce, at every prime, a border of dimension at most twice the number of non-root terminals, or a certificate that every terminal quotient is zero modulo that prime. No prime is discarded solely because its interior rank differs from the rational rank.

Border to related quotients. A fixed-size constraint matrix represents every anchored terminal partition. A merge is a rank-at-most-two update. Maintaining left and right normalizing matrices handles the singular cases in $O(b^2)$ field work per merge, including copying the live state for safe branching.

Modular values to certified integers. An explicit spanning-tree bound controls signed reconstruction. Certificates bind the graph, terminal labels, interior reduction, endpoint normal form, and reconstructed cofactor. The verifier is polynomial-time but deliberately *not* advertised as a quadratic-time verifier.

The delivered implementation realizes these interfaces. A small adapter calls the maintained geometry interface before interpreting a cofactor as a completion observation. This adapter has protocol-fixture tests; execution against a complete pinned upstream checkout remains an integration task, explicitly not part of the reported validation.

1.3 Relation to existing mathematics

Boundary response matrices and spanning-forest partitions have a substantial literature. Kenyon–Wilson, for example, express planar grove connection probabilities using boundary measurements [6]. Weighted matrix-tree formulas, Schur elimination, low-rank matrix identities, and rank-revealing equivalence are standard ingredients. This report does not claim to have invented any of them.

Dynamic rank is itself an active algorithmic subject. The 2026 paper of van den Brand, Kumar, and Zhang gives substantially more sophisticated rank-sensitive bounds in entry- and column-update models [5]. The present primitive instead maintains explicit dense normalizing matrices under a supplied dense rank-one update, with a simple deterministic quadratic bound and an independent verifier. No claim of a new state-of-the-art general dynamic-rank algorithm is made. The contribution relative to the reviewed ProveIt implementation is the exact fixed-dimension terminal-merge representation, its singular-safe modular realization, the associated cost and reconstruction contracts, and the tested software path.

The hierarchy route to unknot recognition remains separate. Lackenby’s July 2026 paper presents a hierarchy-based algorithm and distinguishes bounds on its number of steps from sufficiently precise bounds on the running times of those steps [4]. Faster determinant observations do not fill that hierarchy-encoding gap. Nor do they replace a complete unknot detector such as the Khovanov-homology criterion of Kronheimer–Mrowka [7].

1.4 Notation and complexity model

A graph has $v \geq 1$ vertices and m explicitly listed integer-weighted edges, allowing parallel edges, zero weights, and loops. A nonempty ordered terminal list has size b , with terminal index 0 designated the root. Put

$$t = b - 1, \quad h = v - b.$$

A partition has one root block and q other blocks. At a prime p , the interior nullity is r_p , usually abbreviated r . The border dimension before partition constraints is $d = r + t$.

Field-operation bounds count addition, multiplication, inversion of a nonzero element, comparisons, and ordinary indexed operations. They do not assume a unit-cost integer of arbitrarily large size. Section 8 supplies bit-size accounting. A *supplied merge tree* specifies its parent–child structure and merge labels; finding such a tree from an unordered query collection is a separate cost. All complexity statements charge the initial construction of the explicit matrices.

2 Signed graph quotients and a uniform reconstruction bound

2.1 Cofactor convention

For an edge $e = uv$ of weight $w_e \in \mathbb{Z}$, let

$$L_e = w_e(e_u - e_v)(e_u - e_v)^\top.$$

A loop contributes zero. The signed Laplacian is $L = \sum_e L_e$. Negative weights are permitted; in particular, positive-semidefiniteness is never assumed.

Let π be a partition of the terminal indices. Identify terminal vertices in each block, leaving all interior vertices distinct. An edge whose endpoints become equal is a loop and contributes zero in the quotient Laplacian. Delete the row and column of the root block. We write

$$\tau_\pi = \det L_\pi^* \in \mathbb{Z}$$

for the resulting cofactor. The determinant of the empty matrix is one. This convention covers a quotient with one vertex, and avoids adding an implicit crossingless component to an algebraic computation.

Lemma 2.1 (Weighted matrix-tree formula with signed weights). *For every terminal partition, τ_π equals the sum, over spanning trees of the quotient multigraph, of the product of their edge weights. In particular the cofactor is zero for a disconnected quotient with more than one vertex.*

Proof. Orient the non-loop edges arbitrarily and delete the root row from their incidence matrix, obtaining D . Then $L_\pi^* = DWD^\top$, where W is diagonal with entries w_e . Cauchy–Binet expands its determinant as

$$\det(DWD^\top) = \sum_{S: |S|=s} \det(D_S)^2 \prod_{e \in S} w_e,$$

where s is the number of non-root quotient vertices. A square reduced incidence matrix has determinant ± 1 when its edges form a spanning tree: delete a leaf and induct. If the edges contain a cycle, their incidence columns are dependent; if they fail to connect all vertices, a nonzero vector constant on one non-root connected component annihilates their columns. Thus the other determinants vanish. This proof is a polynomial identity and does not require positive weights. The empty case contributes the empty tree of weight one. $\square$

Proposition 2.2 (One bound for every quotient). *Define*

$$B = \prod_{e \text{ not an original loop}} (1 + |w_e|).$$

Then $|\tau_\pi| \leq B$ for every terminal partition. For an explicit ± 1 edge list this gives $B \leq 2^m$. If a quotient has $h + q + 1$ vertices and all nonzero edge weights have absolute value one, the additional query-specific bound

$$|\tau_\pi| \leq \binom{m}{h+q}$$

holds, with the usual zero convention when $h + q > m$.

Proof. Taking absolute values in Lemma 2.1 bounds the tree sum by the sum of $\prod_{e \in S} |w_e|$ over all subsets of the original non-loop edge list. That sum is the displayed product. Quotient loops merely remove possible subsets. A tree on $h + q + 1$ vertices uses $h + q$ edges, giving the second bound. $\square$

Parallel edges can first be combined by summing their weights; their Laplacian is unchanged. The product bound recomputed on the aggregated graph is still valid and may be much smaller. The adapter’s `SignedGraph.from_laplacian` performs this implicit aggregation. It does not restore the crossing-phase information lost when opposite signed edges cancel; that phase remains the responsibility of the maintained geometric caller.

3 A prime-specific singular interior kernel

Ground the original root terminal and order the remaining vertices with interiors first. Over $\mathbb{F}_p$ write

$$L^* = \begin{pmatrix} A & B_0 \\ E_0 & D_0 \end{pmatrix}, \quad A \in \mathbb{F}_p^{h \times h}, \quad D_0 \in \mathbb{F}_p^{t \times t}. \quad (1)$$

Originally $E_0 = B_0^\top$, but the useful elimination is an equivalence, not a congruence. This distinction matters for both signs and implementation.

Lemma 3.1 (Interior normal form). *There are invertible matrices U, V and a nonzero scalar γ such that*

$$UAV = J_\rho := \text{diag}(I_\rho, 0_r), \quad r = h - \rho, \quad \gamma \det U \det V = 1.$$

They can be found in $O(h^3)$ field operations. Define

$$\hat{B} = UB_0 = \begin{pmatrix} B_a \\ C \end{pmatrix}, \quad \hat{E} = E_0V = \begin{pmatrix} E_a & E \end{pmatrix}, \quad S = D_0 - E_a B_a.$$

For a partition π , let R be its $t \times q$ membership matrix: a row is the unit row of its non-root block, or the zero row if its terminal is in the root block. Then

$$\tau_\pi = \gamma \det \begin{pmatrix} 0_r & CR \\ R^\top E & R^\top SR \end{pmatrix} \quad \text{in } \mathbb{F}_p. \quad (2)$$

Proof. Full-pivot row and column elimination reduces A to J_ρ , keeping the product of determinant changes. This costs cubic field work, and the tracked reciprocal determinant product is γ .

Before elimination, the quotient cofactor is

$$\begin{pmatrix} A & B_0 R \\ R^\top E_0 & R^\top D_0 R \end{pmatrix}.$$

Multiplication on the left by $\text{diag}(U, I_q)$ and on the right by $\text{diag}(V, I_q)$ multiplies its determinant by $1/\gamma$. In the transformed matrix the first ρ interior coordinates form an identity block. Subtracting multiples of those rows and columns eliminates their couplings by determinant-one operations. The remaining block is exactly the matrix in (2), and the eliminated identity block has determinant one. $\square$

Corollary 3.2 (Nullity obstruction and uniform border size). *If $r > q$, then $\tau_\pi = 0$ in $\mathbb{F}_p$. In particular, if $r > t$, every terminal quotient vanishes modulo p . Otherwise all queries admit the fixed base matrix*

$$K = \begin{pmatrix} 0_r & C \\ E & S \end{pmatrix} \quad \text{of dimension } d = r + t \leq 2t. \quad (3)$$

Proof. The first r rows of the border in (2) are supported on q columns. If $r > q$, they are dependent. Since $q \leq t$, the first assertion implies the universal-zero assertion. The final bound follows from $r \leq t$. $\square$

No bad-prime exclusion. The nullity r_p may be larger than the rational interior nullity. The construction simply uses the actual rank at p . A prime with $r_p > t$ contributes a certified zero residue rather than an unusable denominator. Rank normalization over $\mathbb{F}_2$ is valid, since neither this argument nor the later update formulas divide by two. This is particularly useful for signed graphs: cancellation can make an interior singular even when its underlying unsigned graph is well connected.

The universal-zero branch. The implementation records the interior normal-form certificate and stores no border when $r > t$. Querying such a field returns zero. If only a later partition satisfies $r > q$, the static query path returns zero immediately. The dynamic reference implementation does not yet prune that whole descendant subtree, although every further coarsening also has zero residue. Such pruning is a safe additional optimization, not an assumption in the measured timings.

Construction cost. Computing the transformed rectangular blocks and their product costs $O(v^3)$ field operations by ordinary multiplication, including cases where h or t is zero. The explicit grounded matrix and normalizing matrices require $O(v^2)$ field elements. No bound independent of interior size is claimed for this setup.

4 Fixed-dimension matrices for all terminal partitions

The small border in (2) changes size when blocks merge. Repeated determinant elimination on that border loses previous work. We now replace dimension changes by equality-constraint rows.

Definition 4.1 (Anchored partition). Each block of a terminal partition has a distinguished member called its anchor. The root block has anchor 0. Other anchors need not be the smallest members of their blocks. Coordinates of K are ordered as its r null directions followed by terminals $1, \dots, t$. For terminal $i > 0$ let e_i be the corresponding unit column in $\mathbb{F}_p^d$, and put $e_0 = 0$.

Let $k_i^\top$ denote original row $r+i$ of K in one-based indexing. For an anchored partition π , define M_π by the following rows. The first r rows are unchanged from K . If a non-root block G_a has anchor a , replace row a in terminal coordinates by

$$\sum_{i \in G_a} k_i^\top.$$

For every non-anchor terminal $i \in G_a$, use the constraint row

$$e_i^\top - e_a^\top.$$

This also covers the root block, with $e_0 = 0$. There is no row for terminal 0.

Theorem 4.2 (Exact fixed-dimension determinant identity). *For every anchored partition and every commutative coefficient ring,*

$$\det M_\pi = \det \begin{pmatrix} 0_r & CR \\ R^\top E & R^\top SR \end{pmatrix}. \quad (4)$$

Consequently, over $\mathbb{F}_p$, $\tau_\pi = \gamma \det M_\pi$. There is no extra sign depending on the block order, anchor choices, or number of identified terminals.

Proof. Make the following change of variables in the columns. For each non-root block with anchor a , set $x_a = z_a$ and $x_i = z_a + z_i$ at every non-anchor i in that block. For the non-anchor members of the root block set $x_i = z_i$. Keep the null-direction coordinates unchanged.

The change-of-variables matrix is $T = I + N$, where N has a one in row i , column a for each non-anchor in a non-root block. Its nonzero rows are non-anchor rows and its nonzero columns are anchor columns, so $N^2 = 0$. A simultaneous permutation places it in triangular form with unit diagonal. Thus $\det T = 1$, over any commutative ring.

In $M_\pi T$, every constraint row is now exactly the unit row at its non-anchor coordinate. An unchanged null row, when restricted to anchor columns, is the corresponding row of CR . A non-root anchor row is the sum of the original rows of its block. Restricted to null coordinates and anchor columns, these rows are respectively $R^\top E$ and $R^\top SR$.

Use the unit constraint rows to clear their columns from all other rows. These operations have determinant one. Finally reorder rows and columns by the same permutation to place null and anchor coordinates first. The two permutation signs cancel. The result is the small border in (4) direct-summed with an identity matrix. Since every operation has total determinant one, the equality follows. $\square$

Example 4.3 (One merger written out). Take $r = 0$, $t = 3$, and

$$K = \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix}.$$

Keeping anchor 1 while merging terminals 1 and 2 gives

$$M = \begin{pmatrix} a+d & b+e & c+f \\ -1 & 1 & 0 \\ g & h & i \end{pmatrix}, \quad \det M = (a+b+d+e)i - (c+f)(g+h).$$

The latter is exactly the determinant of the two-block quotient matrix. The constraint row is essential: simply adding two rows and deleting one without also identifying columns would give a different determinant.

5 An anchored merge is a rank-two update

Suppose source block G_b with anchor $b \neq 0$ is merged into target block G_a with anchor a , and retain a as the anchor of the union. The target may be the root. Define

$$e = e_b - e_a, \quad \chi_b = \sum_{i \in G_b} e_i, \quad w_b^\top = \sum_{i \in G_b} k_i^\top.$$

The vector w_b is computed from *original base rows of K* , not from the current rows of M_π .

Theorem 5.1 (Rank-two contraction formula). *For the new anchored partition π' ,*

$$M_{\pi'} - M_\pi = -e w_b^\top + \chi_b e^\top. \quad (5)$$

In particular the matrix change has rank at most two over every field.

Proof. If $a \neq 0$, its anchor row gains $w_b^\top$. The former source-anchor row changes from $w_b^\top$ to $e_b^\top - e_a^\top$, hence changes by $e^\top - w_b^\top$. Every former non-anchor $i \in G_b$ changes its constraint from $e_i^\top - e_b^\top$ to $e_i^\top - e_a^\top$, a change of $e^\top$. No other row changes.

The first outer product contributes $w_b^\top$ in row a when $a \neq 0$, and $-w_b^\top$ in row b . The second contributes $e^\top$ in all rows belonging to G_b . This gives precisely the listed changes. When $a = 0$, the convention $e_0 = 0$ removes the nonexistent root row and the same calculation applies. $\square$

The result is not an inverse-update formula and has no invertibility hypothesis. Computing w_b directly takes $O(|G_b|d) \subseteq O(b^2)$ field operations. The other vectors take $O(b)$ work. It remains to handle the two rank-one updates even when either intermediate matrix is singular.

Why anchors must be part of a dynamic certificate. Static quotient values depend only on the unanchored partition. Dynamic normalizing matrices depend on the specific row-constraint representation and hence on the retained anchors. For example, labels $(0, 2, 2)$ and $(0, 1, 1)$ describe the same unanchored partition but different fixed matrices. A verifier must either preserve the recorded anchors or explicitly transform the normal form. The package initially exposed this distinction during development; the final verifier decodes actual anchors, checks that each belongs to its block, and has a nonminimum-anchor regression test.

Why symmetry must not be imposed. Even when the initial graph Laplacian is symmetric, separate left and right elimination need not preserve symmetry. More decisively, the equality-constraint matrix M_π is generally nonsymmetric. Forcing symmetric updates into (5), or replacing E by $C^\top$ after arbitrary equivalence, is not justified.

6 Quadratic updates through every singular case

6.1 Invariant and determinant bookkeeping

For a $d \times d$ matrix M over a field, maintain

$$PMQ = J_s := \text{diag}(I_s, 0), \quad \eta \det P \det Q = 1, \quad (6)$$

where P, Q are invertible. Then

$$\det M = \begin{cases} \eta, & s = d, \\ 0, & s < d. \end{cases} \quad (7)$$

An elementary row operation S updates P to SP and η to $\eta / \det S$; a column operation T updates Q to QT and η to $\eta / \det T$. Row and column swaps therefore negate η , additions preserve it, and multiplication of a row or column by $c \neq 0$ divides it by c . The initialization is the full-pivot elimination used for the interior.

Theorem 6.1 (Singularity-safe rank-one maintenance). *Given (6) and columns $u, v \in \mathbb{F}^d$, one can maintain the invariant for $M + uv^\top$ using $O(d^2)$ field operations and $O(d^2)$ space. The construction works in every characteristic, permits arbitrary rank increases and decreases, and never inverts a zero scalar.*

Proof. Compute

$$x = Pu, \quad y^\top = v^\top Q.$$

This takes $O(d^2)$ work. It remains to normalize $J_s + xy^\top$ while multiplying the accumulated elementary operations into P and Q . If either vector is zero, no change is needed. Partition the coordinates into the first s active coordinates and the remaining null coordinates.

Case 1: x has a nonzero null coordinate. Choose such a coordinate, move it to $s + 1$ by a null-row swap, and scale that row to make the corresponding coordinate of x one. Add multiples of this null row to every other row to clear the other coordinates of x . These operations preserve J_s , because the original null row of J_s is zero. They therefore transform the update to $e_{s+1}y^\top$.

The first s rows of the updated matrix remain the active unit rows. Subtract their multiples from row $s + 1$ to remove the active coordinates of y . If the null part of y is zero, the resulting matrix is again J_s , and the rank is unchanged. Otherwise choose a nonzero null coordinate of y , swap its column into position $s + 1$, normalize the new pivot, and add multiples of its column to clear the other null entries in that row. This produces J_{s+1} . There are $O(d)$ elementary operations, each updating a length- d row of P or column of Q .

Case 2: x has no null part, but y does. Transpose the entire invariant: the normalizing matrices become $Q^\top$ and $P^\top$, and the update vectors become y, x . Apply Case 1 and transpose back. Transposition preserves determinants, so η needs no extra sign. The matrix-copy and transpose work is quadratic.

Case 3: both null parts are zero. Only the active block changes, from I_s to $I_s + x_a y_a^\top$. Put

$$\alpha = 1 + y_a^\top x_a.$$

If $\alpha \neq 0$, then

$$(I_s - x_a y_a^\top / \alpha)(I_s + x_a y_a^\top) = I_s.$$

To apply the left factor to P , first compute the row $y_a^\top P_a$, where P_a consists of the active rows of P . Then perform one outer-product correction. Both operations cost $O(d^2)$. The active determinant changes by α , so η is multiplied by α .

If $\alpha = 0$, choose an invertible active matrix S taking x_a to the final active unit vector e_s . It can be built from one swap, one scaling, and at most $s - 1$ row additions. Apply the paired transformations S and S^{-1} to the active rows and columns. They preserve J_s and their determinant factors cancel. The update becomes $e_s \tilde{y}_a^\top$, where

$$(\tilde{y}_a)_s = y_a^\top x_a = -1.$$

Thus the final active diagonal entry is zero. Subtract suitable multiples of the first $s - 1$ unit rows from the final active row to clear its other entries. The result is J_{s-1} . This case has $s \geq 1$ because $y_a^\top x_a = -1$. It again uses only $O(d)$ elementary operations on explicit length- d vectors.

All selected pivot scalars are nonzero by their selection conditions. The three cases exhaust all possibilities. The scalar updates described before the theorem maintain (6), including in characteristic two. $\square$

Not an application of inverse updates across zero. When $s < d$, the matrix has no inverse. Even for $s = d$, the regular formula is used only if $\alpha \neq 0$. The rank-drop case is handled by normal-form operations, not by division by a vanishing determinant. In (5), the two updates are performed successively; correctness does not require either intermediate determinant to be nonzero.

Cost of explicit transformations. The bound concerns an explicit representation of P and Q , whose storage is quadratic. A generic update may change quadratically many stored entries. This observation explains why quadratic work is natural for this representation; it is not a lower bound for every dynamic determinant data structure or for specialized graph families.

6.2 Persistent branching and failure safety

A search may need to explore several coarsenings of the same partition. The reference implementation stores immutable block sets and clones the normalizing matrices before changing them. A merge constructs both rank-one updates on the child state, and returns the child only after both have completed. The parent remains valid on successful return, on a work-budget exception, or on allocation failure.

Proposition 6.2 (Transactional merge contract). *Assume a parent cursor satisfies the graph, partition, and normal-form invariants. A call to its persistent merge operation either returns a child satisfying the invariants for the merged partition, or raises without changing the parent. The copying overhead is $O(b^2)$ per prime and does not change the asymptotic merge bound.*

Proof. The child holds new dictionaries and new copies of the two normalizing matrices. All row and column operations mutate these copies only. The shared base kernel is read-only. Theorems 5.1 and 6.1 prove the child invariant after the second update. If either update or an intervening allocation raises, the new object is not published and the original matrices and block sets are unchanged. Copying two $d \times d$ matrices costs $O(d^2)$, with $d \leq 2(b - 1)$. $\square$

The low-level `DynamicRank.update` method is mutable and is *not* failure-atomic by itself. Callers requiring the proposition must use a cloned state or the persistent cursor API. Likewise, the budget is an instrumented allowance and deadline check, not an operating-system preemption guarantee. The high-level static query and prime-generation routines are not advertised as fully interruptible production services.

7 Edit-sensitive batch bounds and the partition tree

Definition 7.1 (Merge tree). A merge tree is a rooted tree whose root is the discrete terminal partition. Each edge names two live block anchors and merges its non-root source into its target. Some nodes are designated observations; repeated observations of a node are allowed. Let J be the number of tree edges and Q the number of requested values.

Theorem 7.2 (Per-prime batch complexity). *For a supplied merge tree, all Q signed quotient residues can be computed deterministically in*

$$O(v^3 + Jb^2 + Q) \quad (8)$$

field and ordinary indexed operations per prime, including explicit interior preparation, root normal-form construction, matrix copying, and all merge updates. When $r > t$, the field contributes only a universal zero and no dynamic border is needed.

A depth-first implementation needs $O(v^2 + b^3)$ field elements for matrices and live states. Storing the explicit tree and requested results adds $O(J + Q)$ indexed words. Writing a fresh length- b partition label at every observation adds $O(Qb)$ output work.

Proof. Lemma 3.1 and ordinary rectangular multiplication give $O(v^3)$ preparation. Initialization of the root normal form costs $O(d^3)$ and is absorbed because $d \leq 2v$. For an edge, form the vectors in (5), clone the parent, and apply two updates from Theorem 6.1. These operations total $O(b^2)$. Reading the determinant at an observation uses (7) and multiplication by γ , a constant number of field operations.

Every merge decreases the number of blocks, so the depth is at most $b - 1$. A depth-first traversal keeps at most b normal-form states, each of size $O(b^2)$, besides the shared interior data. Ordinary arrays can represent the supplied child lists. The universal-zero case follows from Corollary 3.2. $\square$

What is being improved. After the same interior preparation, an independent static query forms the smaller border of dimension $r + q$ and eliminates it. A straightforward conservative bound is $O(b^2 + (r + q)^3)$ per query, or immediate zero when $r > q$. Thus the meaningful comparison is not with repeatedly eliminating the full graph Laplacian, but with

$$O\left(v^3 + \sum_{\pi \text{ queried}} [b^2 + (r + q_\pi)^3]\right). \quad (9)$$

For a shallow tree with $J = \Theta(Q)$ and $q_\pi = \Theta(b)$, the dense boundary-elimination contribution changes from cubic to quadratic per edge. This is an improvement of upper bounds for these explicit implementations, not a claim that every such graph has an inherent cubic static complexity.

For deeply coarsened partitions, q_π becomes small while the dynamic matrix retains its original dimension. The smaller static border can then be preferable. Similarly, many unrelated partitions can require J nearly $Q(b - 1)$, eliminating the expected advantage. An edit-sensitive theorem is not a uniform speedup theorem.

7.1 An exact tree on all set partitions

Let $\mathcal{B}_b$ be the b th Bell number. Root every block at its smallest terminal. The following elementary observation removes duplicated intermediate work when all partitions really are requested.

Proposition 7.3 (Bell-tree organization). *All terminal partitions can be organized into a merge tree with exactly $\mathcal{B}_b$ vertices and $\mathcal{B}_b - 1$ edges. Its generation and per-prime evaluation together can be performed in*

$$O(v^3 + \mathcal{B}_b b^2)$$

operations, aside from integer output and any topological admissibility filtering.

Proof. For a non-discrete partition choose its non-singleton block with largest anchor, and split off that block's largest member as a singleton. This gives a unique parent with one more block. Repeating eventually reaches the discrete partition; there are no directed cycles. Thus these parent relations form a tree on all set partitions.

Every edge in the forward direction merges a singleton source $\{c\}$ into a block with anchor $a < c$, retaining a . To enumerate children of a given partition, precompute the largest non-singleton-block anchor and the largest member in each block. A pair (a, c) is admissible exactly when $\{c\}$ is a singleton, a is at least the largest existing non-singleton anchor, and c is larger than every member already in the target block. The initial maximum may be taken as -1 . These tests identify precisely the children whose parent operation is the specified split.

There are at most b^2 candidate pairs at a node, and all its block maxima can be prepared in $O(b)$ operations. Apply Theorem 7.2 along the resulting $\mathcal{B}_b - 1$ edges. This proves both generation and evaluation bounds. $\square$

The delivered `MergePlan.compile` uses canonical ordered merge sequences and shares equal prefixes. When fed all seven-terminal partitions, its trie has exactly 876 edges and 877 observation nodes, agreeing with the proposition. Its practical dictionary-based compilation is distinct from the explicit child-enumeration construction in the proof. The primary complexity theorem accepts a supplied tree; the cost of producing one from arbitrary unsorted labels must not be silently omitted.

The partition count itself remains large. The elementary upper bound $\mathcal{B}_b \leq b^b$ suffices for the quasi-polynomial regime in Section 11. A matching exponential-order lower bound follows by counting pair partitions for even b : $(b-1)!!$ already has logarithm $\Theta(b \log b)$. Accordingly, sharing all intermediate merges does not turn unrestricted Bell-number enumeration into a polynomial algorithm.

7.2 A possible refinement: zero descendant cones

Fix a prime. Once a node has $q < r_p$, every descendant also does, so the entire descendant cone has zero residue. This permits pruning the field computation independently at each prime. Such zeros must still be included in signed reconstruction. The present static query uses the pointwise obstruction, while the measured dynamic code retains its state except in the universal-zero case $r_p > t$. A future implementation can obtain a sharper bound by counting only edges above these prime-specific zero cones. That refinement changes work, not the mathematical answer or the required reconstruction modulus.

8 Signed reconstruction and a deterministic bit budget

8.1 Exact reconstruction

Let $p_1, \dots, p_a$ be distinct primes, $P = \prod_i p_i$, and let each residue be computed using the actual interior rank at that prime.

Theorem 8.1 (Signed Chinese-remainder certificate). *If $P > 2B$, there is at most one integer $z \in [-B, B]$ with the computed residues. The integer quotient cofactor τ_π is that integer. Reconstructing the residue $x \in [0, P)$ and taking*

$$z = \begin{cases} x, & 2x \leq P, \\ x - P, & 2x > P \end{cases}$$

recovers τ_π exactly. The final check $|z| \leq B$ rejects inconsistent residue data.

Proof. Two such integers differ in absolute value by at most $2B < P$. If they have equal residues, their difference is a multiple of P and must be zero. Proposition 2.2 supplies existence for the genuine graph cofactor. Since $B < P/2$, its representative lies on the stated side of the midpoint. $\square$

Example 8.2 (The mixed-sign unit trap). Consider a graph with two vertices and one edge of weight 41. Its cofactor is 41, and the uniform bound is $B = 42$. At primes 3, 5, 7 its residues are respectively $-1, +1, -1$. Their product modulus is $105 > 2B$, yet the integer has absolute value 41, not one. Thus “each modular determinant is ± 1 ” is not a unit certificate, even under the usual signed-reconstruction modulus bound. The signs cannot be chosen independently and then discarded.

A single residue outside $\{1, -1\}$ excludes an integer cofactor of absolute value one. Conversely, to use the congruences $z^2 \equiv 1 \pmod{p_i}$ without reconstructing the signed value would require a bound for $z^2 - 1$, of order B^2 rather than B . Signed CRT avoids that unnecessarily larger bound. This is a guard for new modular implementations, not an assertion that the reviewed rational implementation contains such an error.

8.2 Elementary control of the prime supply

Define $H = \lceil \log_2(B + 1) \rceil$. The implementation enumerates small primes by exact trial division until their product exceeds $2B$. The following coarse bound is enough to certify polynomial bit complexity without a delicate prime-gap assumption.

Lemma 8.3 (Polynomial prime budget). *The required prefix has at most $H + 2$ primes. Each of those primes is at most $16(H + 2)^2$. The final product has $O(H + \log(H + 2))$ bits. Trial-division generation takes polynomial time in H .*

Proof. Any $H + 2$ distinct primes have product at least $2^{H+2} > 2B$, proving the count bound once those primes have been found.

Let $N = 16(H + 2)^2$, an even integer, and put $k = N/2$. The central binomial coefficient is at least $2^N/(N + 1)$. For every prime p , Legendre’s formula gives

$$v_p \binom{2k}{k} = \sum_{j \geq 1} \left(\left\lfloor \frac{2k}{p^j} \right\rfloor - 2 \left\lfloor \frac{k}{p^j} \right\rfloor \right) \leq \lfloor \log_p N \rfloor,$$

because each summand is zero or one. If $\pi(N)$ counts primes at most N , it follows that

$$\frac{2^N}{N+1} \leq \binom{N}{N/2} \leq N^{\pi(N)}.$$

Since $N \geq 6$, $\log_2(N+1) \leq N/2$, so

$$\pi(N) \geq \frac{N}{2 \log_2 N}.$$

Also $\log_2 N = 4 + 2 \log_2(H+2) \leq 4(H+2)$. Thus $\pi(N) \geq 2(H+2)$, more than enough.

Immediately before the last selected prime, the product is at most $2B$; multiplying by a prime at most N gives $P \leq 2BN$. Finally, testing every candidate at most N by trial division up to its square root uses $O(N^{3/2}) = O((H+2)^3)$ divisions of $O(\log(H+2))$ -bit integers. This deliberately loose bound is polynomial. $\square$

8.3 Complete bit accounting

For ± 1 edge inputs, $H \leq m+1$. For integer weights, H is bounded by a constant plus the sum of their binary lengths. Let ℓ be the complete encoded graph length, including labels and even loop weights that have no effect on the Laplacian.

For a supplied tree, the dominant algebraic work is at most

$$(H+2) O(v^3 + Jb^2 + Q) \tag{10}$$

operations on fields of $O(\log(H+2))$ -bit size. Indexing and unreduced dot-product accumulation add logarithms of matrix and tree sizes, not exponentially many bits. Sequential CRT keeps $O(H + \log(H+2))$ -bit accumulated integers. A coarse schoolbook bound of $O((H+2)^3)$ bit work per value, with additional logarithmic factors, safely covers its modular reductions, inversions, and the reference implementation's repeated small-prime checks.

For clarity, with

$$L = \log_2(v + J + Q + H + 3),$$

one conservative deterministic implementation bound is

$$O\left(\ell + (m + v^2)(H+2)^2 + (H+2)(v^3 + Jb^2 + Q)L^4 + (Q+1)(H+2)^3 L^4\right). \tag{11}$$

The intentionally generous logarithmic power covers ordinary exact scalar arithmetic and indexed bookkeeping. Computing the product bound, repeated conversion of integer Laplacian entries to each field, prime generation, and reconstruction are included; compilation of an arbitrary unsorted query collection is not. An array/ordered-map implementation of the prescribed tree traversal gives the deterministic accounting. The Python compiler uses hash tables for practical prefix sharing; its measured performance is not presented as a worst-case hashing theorem.

The eager exact reference observer retains every prime's kernel. Its matrix storage therefore has the extra factor $a \leq H+2$. A sequential-prime variant could reuse one kernel's workspace and store only the accumulated CRT outputs, but is not the measured implementation. This distinction is significant when the graph interior is large.

Verification is a separate expense. Writing endpoint normal forms already requires quadratic output per endpoint and per prime. Deterministically checking a supplied normal form by direct matrix multiplication and determinant elimination costs cubic work. Including verification of every endpoint can therefore remove the quadratic *query-time* advantage. The package offers independently checkable evidence; it does not hide verifier cost inside a claim of quadratic certified end-to-end execution.

9 Certificates and the boundary-geometry interface

9.1 An independently checkable algebraic certificate

For one endpoint, the certificate contains the integer graph, ordered terminal list, actual anchored labels, uniform bound, signed claimed value, and one record per prime. A prime record contains the interior rank normal form (U, V, ρ, γ) and, unless the entire field is zero by $r > t$, the endpoint normal form (P, Q, s, η) .

Theorem 9.1 (Soundness of the algebraic verifier). *A verifier that performs the following checks accepts only the true signed cofactor of the supplied terminal quotient:*

- (i) *validate the graph, distinct terminals, anchored partition, and independently recompute B ;*
- (ii) *check distinct prime moduli, $\gamma \det U \det V = 1$, and $UAV = J_\rho$;*
- (iii) *rebuild the border from the supplied normalizing matrices and the original grounded Laplacian, then rebuild M_π from the anchored labels;*
- (iv) *either check $r > t$ and the universal-zero marker, or verify $\eta \det P \det Q = 1$ and $PM_\pi Q = J_s$;*
- (v) *take each residue as zero when forced or rank-deficient, otherwise $\gamma\eta$, and verify the signed CRT value with $P_{\text{CRT}} > 2B$.*

Proof. The determinant-product identities imply invertibility of all asserted basis transformations. The matrix identities then establish the claimed ranks; this is important because a zero determinant alone would not justify a supplied nullity. Lemma 3.1 establishes the reconstructed border formula. Theorem 4.2 binds the endpoint constraint matrix to the precise terminal partition. Equation (7) gives its determinant from the checked normal form, including the zero-rank cases. Theorem 8.1 then binds the uniquely reconstructed signed integer to the graph cofactor. $\square$

The package’s verifier rebuilds these matrices and uses static elimination and matrix multiplication. It never calls the dynamic update method. Independent integer Bareiss determinants supply an additional audit path. These are meaningful implementation cross-checks, but they still share elementary Python arithmetic and some small utility functions; no claim of a separately formalized trusted kernel is made.

The current single-endpoint JSON schema repeats interior data in separate endpoint files. A production batch schema should store one interior certificate per stage and prime, then reference it from many endpoints. Merely putting the same interior certificate in every query file has $O(Qv^2)$ evidence size per prime and should not be called boundary-only evidence.

9.2 A fully explicit certificate example

Take the six-cycle, all weights +1, with terminals $(0, 1, 2)$. Its remaining interior vertices form a three-vertex path with tridiagonal interior matrix of determinant four. The interior is singular modulo two but nonsingular modulo 3, 5, 7.

The discrete terminal partition has cofactor six. Merging terminal 1 into terminal 2, retaining the nonminimum anchor 2, gives labels $(0, 2, 2)$ and cofactor five. Merging that block into the root gives labels $(0, 0, 0)$ and cofactor four. Here $B = 64$ and the prime product $2 \cdot 3 \cdot 5 \cdot 7 = 210$ exceeds 128. The package includes and independently verifies a JSON certificate for each of these values. This example simultaneously exercises a modular rank jump, a nonminimum anchor, and a root merge; it is not being presented as a hard knot instance.

9.3 What the maintained geometry must still certify

The reviewed `BoundaryTait.partition` method checks boundary coverage, compatibility with the inherited checkerboard coloring, and the Euler-characteristic condition for a spherical completion. Its vertices are *cut-face fragments*, not unsplit regions of the original diagram. Replacing fragments by original regions can incorrectly identify faces before a new boundary closure is chosen [2].

For accepted connected projection completions, the maintained observer’s convention is

$$J_{\text{raw}}(i) = (-i)^{\beta+v_{\pi}-1}\tau_{\pi}, \quad (12)$$

where β is its crossing-sector sum and v_{π} is the completed black-vertex count. We use β here to avoid confusing that sum with the reconstruction bound B . Projection-disconnected completions use the maintained zero branch. The modular adapter preserves this convention, obtaining its phase and connectivity metadata directly from the geometry object rather than trying to infer them from the Laplacian alone.

A zero edge, a cancelled pair of signed parallel edges, or a loop may be invisible to the Laplacian while still contributing to the crossing phase. This is why the graph certificate does not stand alone as a knot or link certificate. It proves the cofactor of the supplied graph quotient. The correspondence between that graph, a classical completion, and an invariant observation remains a separate obligation.

9.4 Adapter status and deliberate limitations

The supplied `integration/boundary_adapter.py` accepts an existing object with the reviewed `BoundaryTait` interface. Its single-query path validates the geometry and reconstructs the exact cofactor. Its batch path validates all requested completions, compiles their terminal partitions into a merge trie, and then evaluates the trie. Disconnected projections retain the existing zero convention.

This is an additive research adapter, not a production replacement. In particular:

- the reviewed upstream geometry was not independently reimplemented and its complete repository test suite was not run here;
- the delivered adapter tests use an explicit protocol fixture, not an upstream knot corpus;
- prime preparation and all static paths do not yet expose every production deadline or local-inference budget;
- no new recognition verdict is introduced, and no existing recognizer default is changed.

The first integration experiment should therefore compare complete accepted-query streams against the current rational kernel before dispatch is changed. The package contains a script for that comparison and records that it was not executed in this research environment.

10 Validation and performance measurements

10.1 Correctness checks that were actually run

The standard-library Python package was executed on Python 3.13.5 under the Linux environment recorded in the machine-readable results. It passes 39 unit-test methods. These include dimension-zero conventions, characteristic two, all rank-change cases, integer and modular agreement, weighted parallel edges and loops, malformed partitions, root merges, nonminimum anchors, certificate tampering, the mixed-sign CRT trap, and transactional budget failures.

The separate reproducible audit performs the following checks.

Audit item	Completed checks
Rank-one matrix updates, with static determinant and normal-form verification	19,200
Prime-specific signed-graph kernels	600
Kernels with singular interior	193
Kernels with universal-zero residue	76
Signed-graph quotient values versus independent integer Bareiss elimination	1,772
Observed endpoint rank changes in graph merge chains	546
All set partitions of seven terminals	877
Distinct merge edges in their compiled trie	876
Exported exact endpoint certificates	3

The matrix audit uses primes 2, 3, 5, 101, dimensions one through eight, six initial states per size and prime, and 100 updates per state. The branch counts are 1,774 zero updates, 3,876 left-null-tail cases, 1,111 transposed right-tail cases, 9,847 regular active-block cases, and 2,592 rank drops. Thus the successful comparisons are not confined to invertible matrices or to one characteristic.

The graph audit uses 150 seeded weighted graphs on one through ten vertices, with four prime kernels per graph and random terminal lists. At every merge-chain node it compares the dynamic residue, the static smaller-border residue, and an independently rebuilt integer quotient determinant. The latter constructs the quotient directly from the original weighted edges, without using the terminal kernel. Every live normal-form certificate is checked separately.

The seven-terminal audit is algebraically exhaustive over partitions of one selected signed wheel graph, not exhaustive over all graphs or all knot prefixes. Finite checks support the reference code; the theorems supply the general algebraic statements.

10.2 Benchmark design

The field benchmarks use the prime 1009. Every timed arm includes a fresh interior kernel. The dynamic arm additionally includes root normal-form setup, depth-first state copying, and both rank-one operations at each merge. The static arm independently eliminates the *smaller* singular-safe border at each query. An identical static control arm measures ordinary timing variation.

Each workload has one excluded warmup per arm and seven shuffled paired rounds. The displayed times are medians of complete arm times. Correctness comparison occurs after the timing interval; graph and merge-tree discovery occur before all arms. The JSON file retains separate setup and query times, every raw sample, random seed, exact graph and merge events, environment information, and a digest of the independently checked answers. Ratios are static time divided by dynamic time, so a value greater than one favors dynamic evaluation.

The wheel workloads have b rim terminals and one interior hub. The shallow merge tree first merges one of at most eight disjoint adjacent terminal pairs, then optionally a second pair. Thus most quotients retain nearly the full boundary dimension. The grid workloads use a triangulated $w \times w$ square grid, with its outer boundary as terminals; these have nontrivial interiors. The signed-wheel chain repeatedly merges another terminal into the root. These are explicit planar *graph-observer* workloads, not recorded recognizer traces and not a constructed family of difficult knots.

The largest shallow wheel improves by a factor of 3.34. The two grid workloads improve by factors 1.29 and 1.55, so the observed benefit is not limited to an all-terminal graph or to zero-cost interior elimination. On eight terminals the dynamic method is slower, with ratio 0.62. These differences match the algorithmic expectation that quadratic updates need enough boundary work to amortize normal-form maintenance and copying.

Workload	b	Q	Static (ms)	Dynamic (ms)	Ratio	Control
wheel 8 shallow	8	11	0.40	0.65	0.62	1.00
wheel 16 shallow	16	37	5.88	5.51	1.07	0.99
wheel 32 shallow	32	37	40.38	23.52	1.72	0.98
wheel 64 shallow	64	37	277.57	92.82	2.99	0.99
wheel 96 shallow	96	37	919.17	275.16	3.34	1.00
grid 6 shallow	20	37	12.23	9.48	1.29	0.96
grid 10 shallow	36	37	82.88	53.61	1.55	1.01
signed wheel 64 chain	64	64	150.21	116.91	1.28	0.98

Table 1: Finite-field observer measurements, fresh setup included. “Control” is the static/control ratio. No whole-recognizer timing is represented.

The signed 64-terminal chain still gives a factor 1.29 in this particular experiment; it does not contradict the warning about heavily coarsened queries. The total includes early large-border queries, and the specific signed matrices have their own zero and sparsity patterns. It is not evidence that retaining a fixed-size matrix always wins after the small border has nearly disappeared.

10.3 Exact arithmetic gives an important negative result

Two additional workloads execute the complete eager small-prime observer and signed reconstruction. We compare its dynamic and static versions with direct integer quotient construction followed by Bareiss elimination.

Workload	Primes	Queries	Modular static	Modular dynamic	Integer
Signed wheel, $b = 6$	6	7	1.142 ms	2.106 ms	0.137 ms
Signed wheel, $b = 10$	8	16	7.820 ms	9.021 ms	0.814 ms

Table 2: Complete exact observer comparisons, including setup. Direct integer arithmetic is the fastest method on these small cases.

The dynamic exact observer is about 15.3 and 11.1 times slower than direct Bareiss elimination on these two small workloads. The conservative product bounds require six and eight primes, while the direct determinants involve small matrices and modest integers. The identical static control differs by a factor of 1.29 in the smaller exact case and 1.07 in the larger one, so submillisecond timing differences should not be overinterpreted. The much larger gap to direct integer arithmetic argues against an eager modular default on small input, even though the modular method has useful exactness and bit-complexity guarantees.

An initial complete benchmark is retained separately in `results/benchmark_initial.json`; it gave a largest-wheel ratio of 3.48, versus 3.34 in the final case-by-case replay. An interrupted intermediate rerun is labeled as partial and is not the source of either table. No whole-recognizer speedup, representative knot-corpus advantage, or favorable full-kernel integration result is claimed. In particular, the earlier maintained boundary-reuse timings and its reported query audit were not rerun or incorporated into the new timing table. They motivated this study but are not new experimental evidence [2].

10.4 Interpreting performance without overfitting

The field measurements support the narrow hypothesis that related, nearly full-boundary queries can benefit from shared rank normalization. They do not identify an optimal dispatch threshold. A useful dispatcher needs at least the boundary dimension, actual modular nullities, number and shape of anticipated queries, coefficient bound, and expected ratio of shared merge edges to observations. A single terminal-count threshold is unlikely to cover both small exact integer cases and large field-query batches.

The benchmark graph families are intentionally transparent and replayable. Their simplicity is a strength for checking algebra and a weakness for predicting recognition speed. A maintained integration should preserve the same exact answer comparisons and paired controls while substituting actual completion streams and including stream collection, plan compilation, geometrical validation, and the fallback recognizer in separately labeled measurements.

11 Consequences for the quasi-polynomial goal

Corollary 11.1 (Quasi-polynomial observer regime). *Let n be an external input-size parameter. Suppose v , the integer graph encoding length, and the number of stages are polynomially bounded in n . If the total supplied merge-tree size and number of observations are at most*

$$\exp(O(\log^2(n+2))),$$

then the complete exact observer computation described here has that same quasi-polynomial bit bound. Finding the stage graphs and their merge trees must also fit the claimed budget.

In particular, evaluating all terminal partitions at each stage has a quasi-polynomial bound whenever

$$b \log(b+2) = O(\log^2(n+2)). \quad (13)$$

Proof. The input-length assumption bounds H polynomially in n , and $b \leq v$. Every factor in (11) other than the tree and output counts is polynomial in n and logarithms of those counts. A polynomial number of such stages remains quasi-polynomial. For the final assertion, $\mathcal{B}_b \leq b^b$, so Proposition 7.3 and (13) give the required tree-size bound. $\square$

A sufficient asymptotic example of (13) is $b = O(\log^2 n / \log \log n)$ for large n . If only a small selected family of completions is queried, the edit-sensitive condition can hold at much larger boundaries. Conversely, a small border dimension alone does not bound the number of times an observer is invoked by a homological scan.

The corollary is an observer theorem. It computes exact signed cofactors. It does not assert that all set partitions are classical boundary completions. It does not equate Bell-number partition families with Catalan matching families. It does not control Khovanov generator multiplicities, surviving chain objects, or the number of branches in a topological search. The topological admissibility filter and the process generating the observations must be analyzed on their own terms.

What would transfer the bound to recognition. A complete recognition proof based on this component needs a sound and complete certificate/search framework, a bounded generation procedure for all observations it actually requires, affordable transitions between stages, and a quasi-polynomial cost for every remaining operation, including inconclusive branches. A cofactor

equal to the unknot’s value is only an inconclusive scalar observation unless an independent theorem makes the surrounding certificate complete. The adapter supplies no such theorem.

The maintained homological and hierarchy approaches therefore retain their existing unresolved complexity obligations. In the homological path, bounding the allocated and surviving state counts remains essential, as the corridor analysis already emphasizes [3]. In the hierarchy path, the encoding size and cost of the topological operations are not supplied by a graph determinant kernel. A polynomial improvement inside an unbounded number of queries is useful engineering and algebraic progress, but not a proof that the number of queries is small.

A precise performance target for the next stage. For actual suffixes s , let v_s, b_s, H_s, J_s, Q_s be the parameters above. The relevant observation-work proxy is

$$\sum_s (H_s + 2)(v_s^3 + J_s b_s^2),$$

with reconstruction and query-generation costs added. This is more informative than maximum boundary width alone. To improve the global asymptotic class, the next structural theorem must bound this sum *and* the non-observer search work on every input in the claimed class. The present report proves the local cost formula, not the needed global structural inequality.

12 Further research questions

The following problems are intended as concrete continuations rather than implied consequences of the current experiments.

12.1 Actual-query merge complexity

For the accepted completion streams generated by the maintained scanner, how large is the minimum rooted merge representation compared with the number of distinct terminal partitions? Measure the canonical compiler’s J/Q , compare alternative anchor choices and traversal orders, and distinguish genuine shared prefixes from equality-cache hits. The first useful dataset is a replay of the existing boundary-query audit, with no change in recognizer policy. A structural bound on J for realized scan states would be more consequential than another constant-factor linear-algebra optimization.

12.2 Adaptive static, integer, and dynamic dispatch

Can a policy select among direct integer elimination, static modular borders, and dynamic normal forms using certified setup and remaining-work estimates? The small exact negative results rule out an unconditional modular default. A useful theorem would compare against the best of these modes *including* lost setup work and query-plan construction. A generic appeal to an adaptive heuristic is not a competitive bound, especially when q shrinks while fixed dimension d stays large.

12.3 Zero-cone pruning across primes

Implement the descendant-cone obstruction $q < r_p$, then count how many field-edge updates it actually removes. Different primes can have different cones. Can their description be shared compactly while signed CRT still receives every required zero residue? The proof of Corollary 3.2 makes this optimization safe, but says nothing about its frequency in actual knot suffixes.

12.4 Reuse between successive suffix stages

Removing one crossing changes face fragments, the terminal set, and possibly which coordinates are interior. Is there a bounded-rank or small-border transition theorem for these *geometric* changes, with a replayable correspondence of vertex labels? Reusing a matrix factorization without certifying the new fragment identification is unsafe. A successful transition theorem could save repeated v_s^3 setup, which the present fixed-stage result does not address.

12.5 Cofacial partition structure and grove identities

Which terminal partitions produced by a valid planar scan form a smaller algebraically closed family? The grove/response-matrix literature suggests additional identities [6], but signed weights and singular interiors complicate denominator-based reductions. The target is a concrete compression of the realized observation family, with exact phases and null directions, not simply replacing the word “Bell” by “Catalan” without a geometric proof.

12.6 Cheaper deterministic evidence

Can many endpoint normal forms be certified in substantially less than one cubic verification per endpoint, while keeping a deterministic proof of the signed values? Sharing the interior certificate is straightforward; sharing endpoint checks is not. Randomized matrix-product checks might be useful in testing but would require an explicit error model and would not be the deterministic verifier claimed here. A formalization of the fixed-matrix and rank-two identities is a smaller, well-bounded first proof-assistant target.

12.7 Rank-sensitive and sparse update engines

Can modern dynamic-rank techniques improve the explicit quadratic cost in this specialized update model without making determinant sign and singularity certificates prohibitively expensive? The general entry- and column-update results in [5] do not directly answer that question. Separately, sparse kernels can make fresh elimination much cheaper than its cubic envelope; any claimed improvement must compare with that actual sparse baseline rather than only dense matrix dimensions.

12.8 Sharper reconstruction and early obstruction

The product bound is robust but deliberately loose. Can a cheap stage-wide signed-tree bound, or a query-specific minor bound, reduce the prime count enough to beat direct integer arithmetic on moderate instances? For one-sided obstruction queries, what partial modular evidence suffices for the *surrounding* certificate? Such a result must retain the distinction between excluding absolute cofactor one for a validated whole knot and observing an arbitrary closure of a suffix.

12.9 From scalar observations to homological compression

Can a family of exact completion observations certify that a portion of a relative chain complex is irrelevant, thereby reducing allocated objects rather than merely measuring them more cheaply? A scalar cofactor is not a substitute for a differential or chain-homotopy equivalence. The useful research target is a proved sufficiency theorem for a specified collection of observers, or a counterexample showing that the collection cannot support the intended reduction.

12.10 A global obstruction family or a global structural theorem

Construct realized knot-prefix families for which the necessary observation count or merge complexity is provably large, despite small determinants and simple local geometry. Alternatively, identify a nontrivial input class where all required states, stage transitions, and observation trees satisfy explicit polylogarithmic or quasi-polynomial bounds. Either outcome would sharpen the current gap between local algebraic progress and the general recognition target.

13 Integration and reproducibility

The archive is intended as a new additive directory, for example

Topology/UnknotRecognition/research/dynamic_terminal_20261008/.

It does not overwrite any maintained scanner or report. Its top-level README records the mathematical scope, the code entry points, the benchmark caveats, and the exact commands below.

```
python -m unittest discover -s tests -v
python scripts/audit.py
python scripts/benchmark.py
python scripts/verify_certificate.py certificates/cycle6-nonminimum-anchor.json
sh build.sh
python scripts/verify_manifest.py
```

The first four commands exercise only the delivered package. The benchmark writes fresh samples and therefore changes result-file hashes; it should be run in a working copy before regenerating a release manifest. The final command checks the hashes of the delivered release. No network access or external Python packages are required by the algebra, tests, or benchmark. Rebuilding the PDF requires a conventional LaTeX installation.

The optional upstream replay command is documented separately in `integration/README.md`. It takes an explicit checkout and a JSON completion-query stream. It must validate each accepted geometry query and compare both static and batch modular results with the current rational observer. It was not run in this environment, so its presence in the archive is not a claim of upstream compatibility validation.

Reproduction boundary. The provenance file lists source files reviewed through the GitHub connector and distinguishes them from locally executed code. The package contains no full repository checkout and no redistributed third-party paper or font files. The SHA-256 manifest covers the delivered files except itself. Logs and machine-readable audit and timing data belong to this package, not to a fabricated run of the maintained upstream suite.

14 Conclusion

The main proved improvement is local but definite: a terminal block merger can be realized without changing determinant dimension, with the exact update

$$M_{\pi'} = M_{\pi} - (e_b - e_a)w_b^{\top} + \chi_b(e_b - e_a)^{\top}.$$

A prime-specific singular interior reduction and a quadratic rank-normal-form primitive turn that identity into an executable exact observer. Signed reconstruction and a separate verifier prevent singular primes and independent modular sign choices from becoming correctness gaps.

The experiments identify both a useful regime and a limit. Shared finite-field boundary queries can improve over fresh small-border elimination, while eager exact modular work loses badly to direct integer arithmetic on the tested small cases. Accordingly, the deliverable is an optional, proof-backed component and a research path for exploiting actual query structure. The general quasi-polynomial recognition goal still requires a structural bound on the generated work, not just faster evaluation of one part of it.

A Algorithmic contracts in compact form

A.1 Prime kernel

```

Input: integer graph G; terminals T with root T[0]; prime p
1. Form the grounded Laplacian [A B0; E0 D0] over F_p.
2. Find U*A*V = diag(I_rho, 0), gamma*det(U)*det(V)=1.
3. Set r = number_of_interiors - rho and t = len(T)-1.
4. If r > t, return the interior certificate and ALL_ZERO.
5. Compute Bhat=U*B0, Ehat=E0*V.
6. S=D0-Ehat[:, :rho]*Bhat[:,rho, :].
7. K=[0, Bhat[rho:, :]; Ehat[:, rho:], S].
Output: K, r, gamma, interior certificate.

```

A.2 Persistent merge

```

Input: parent cursor, live anchors a and b, b != 0, a != b
1. Copy the parent's block map and its normalizing matrices.
2. Read source block G_b from the parent, not the updated map.
3. e=e_b-e_a; chi=sum(e_i for i in G_b).
4. w=sum(original K row i for i in G_b).
5. Update the child normal form by (-e)*w^T.
6. Update the child normal form by chi*e^T.
7. Replace G_a by G_a union G_b and remove anchor b.
8. Publish the child only after all checks and updates complete.
Output: a new cursor; parent is unchanged.

```

The implementation changes only unpublished child metadata before the updates; this is equivalent to the presentation above because the vectors are read from the parent and the immutable base.

A.3 Rank-one branch conditions

Condition after $x = Pu, y^T = v^T Q$	Required operation
$x = 0$ or $y = 0$	Leave the normal form unchanged.
x has a null component	Align it using null-row operations; inspect the null part of y for a new pivot.
Only y has a null component	Transpose the invariant, use the preceding case, and transpose back.
Both are active, $1 + y^T x \neq 0$	Apply the active rank-one inverse formula and update the determinant scale.
Both are active, $1 + y^T x = 0$	Active conjugation and one row clearing remove the final active pivot.

B Proof and implementation obligation ledger

Obligation	Status in this contribution
Signed graph cofactor identity	Proved by incidence-matrix Cauchy–Binet; independent integer quotient builder implemented.
Interior singularity at every prime	Proved and implemented by separate left/right equivalence; no denominator-prime rejection.
Fixed-size determinant sign	Proved by a determinant-one column change and paired row/column reordering.
Rank-two merge with arbitrary anchors	Proved entrywise; implemented and tested, including root and nonminimum anchors.
Rank changes during each rank-one update	All cases proved and independently audited against static elimination.
Exact signed reconstruction	Proved with a graph-derived bound; mixed-sign regression included.
Algebraic evidence verification	Implemented independently of the update routine; cubic verifier cost explicitly retained.
Planarity and cut-face correctness	Inherited from reviewed maintained geometry; not independently revalidated on an upstream corpus here.
Whole-recognizer correctness after integration	No production dispatch change supplied; remains an integration obligation.
Whole-recognizer performance	Not measured. Reported gains are graph-observer microbenchmarks only.
General quasi-polynomial state bound	Not established; local observer and restricted-family bounds only.
Formal proof-assistant verification	Not performed.

References

- [1] Vladimir Reshetnikov, *ProveIt, Topology/UnknotRecognition*, repository source review at revision `cbc627dbbba08e3125166447ab3aa1157f9dc42e`, 8 October 2026. <https://github.com/VladimirReshetnikov/ProveIt/tree/cbc627dbbba08e3125166447ab3aa1157f9dc42e/Topology/UnknotRecognition>.
- [2] ProveIt contributors, *Adaptive boundary summaries for repeated completion queries*, `synthesis/boundary_reuse.tex`, and the implementation `fast/determinant_research/boundary_tait.py`, in [1]. Reviewed Git blob identifiers are recorded in `PROVENANCE.json`.
- [3] ProveIt contributors, *Survivor corridors and adaptive transfer direction*, `synthesis/corridor_transfer.tex`, in [1].
- [4] Marc Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, 25 July 2026. See especially Section 9 for the distinction between step counts and running-time estimates. <https://arxiv.org/abs/2607.23350>.
- [5] Jan van den Brand, Vishal Kumar, and Daniel J. Zhang, *Dynamic Rank, Basis, and Matching*, arXiv:2605.09917v1, 11 May 2026. <https://arxiv.org/abs/2605.09917>.
- [6] Richard W. Kenyon and David B. Wilson, *Boundary Partitions in Trees and Dimers*, Transactions of the American Mathematical Society **363** (2011), 1325–1364; arXiv:math/0608422v4. <https://arxiv.org/abs/math/0608422>.
- [7] P. B. Kronheimer and T. S. Mrowka, *Khovanov homology is an unknot-detector*, arXiv:1005.4346, 24 May 2010. <https://arxiv.org/abs/1005.4346>.